

System Design Document (SDD)

1. Introducción
 - 1.1 Propósito del Documento
 - 1.2 Alcance del Sistema
 - 1.3 Público Objetivo
3. Arquitectura del Sistema
 - 3.1 Descripción de Componentes
 - 3.3 Elección de Base de Datos (Supabase / PostgreSQL)
 - 3.4 Elección de Autenticación (Supabase Authentication)
 - 3.5 Elección de Almacenamiento (Supabase Storage vs. Firebase Storage)
 - 3.6 Elección de Servicios de Notificación (Firebase Cloud Messaging)
 - 3.7 Elección de Infraestructura Backend (Supabase Functions vs. Backend Personalizado)
 - 3.8 Elección de Patrón de Diseño: MVVM (Model-View-ViewModel)
 - 3.9 Elección de Escalabilidad (Base de Datos Relacional vs. No Relacional)
4. Diseño de la Base de Datos
 - 4.1 Modelo de Datos
5. Diseño de la Interfaz de Usuario
 - 5.1 Pantallas Principales
 - 5.2 Pantallas segunda iteración
6. Diagramas de secuencia
 - 6.1 Crear evento
 - 6.2 crear evento flujo alternativo
 - 6.3 listar evento
 - 6.4 crear artículo
 - 6.5 crear articulo flujo alternativo
 - 6.6 listar artículo
7. Diagramas de secuencia 2da iteración
 - 7.1 Editar eventos
 - 7.2 Editar artículos
 - 7.3 Hacer pregunta
 - 7.4 Responder pregunta
8. Diagramas de secuencia 3er iteración
 - 8.1 Visualizar detalles de actividad
 - 8.2 Registrar asistencia a evento
 - 8.3 Eliminar evento
 - 8.4 Eliminar producto
9. Mapas de navegación
 - 9.1 Caso de uso 9 - Visualizar detalles de evento

- 9.2 Caso de uso 10 - Registrar asistencia a evento
- 9.3 Caso de uso 11 y 12 - Eliminar evento y eliminar producto
- 10.1 Caso de uso 9 - Visualizar detalles de evento
- 10.2 Caso de uso 10 - Registrar asistencia a evento
- 10.3 Caso de uso 11 - Eliminar evento
- 10.4 Caso de uso 12 - Eliminar artículo

System Design Document (SDD)

1. Introducción

1.1 *Propósito del Documento*

Este documento describe el diseño del sistema para una aplicación móvil enfocada en difundir alternativas ecológicas, promover eventos relacionados con el cuidado del medio ambiente y facilitar la solicitud de ayuda de personas interesadas en estas actividades. Está dirigido a usuarios con poca disponibilidad de tiempo pero con la intención de ayudar.

1.2 *Alcance del Sistema*

La aplicación permitirá a los usuarios:

- Listar y crear eventos ecológicos.
- Listar y crear artículos relacionados con el medio ambiente.
- Editar Eventos
- Editar artículos
- Publicar pregunta
- Responder pregunta
- Ver detalles de actividad
- Registrar asistencia
- Eliminar evento
- Eliminar artículo

1.3 *Público Objetivo*

El sistema está diseñado para personas interesadas en el medio ambiente, pero con limitaciones de tiempo. Está enfocado en usuarios de dispositivos móviles con sistemas operativos Android.

3. Arquitectura del Sistema

3.1 Descripción de Componentes

- **Frontend:** La interfaz de usuario será implementada usando **Flutter SDK versión 3.19.0**, siguiendo el patrón de **Arquitectura por Capas** para mantener una arquitectura limpia y modular.
- **Backend:** Los servicios del backend se manejarán con Supabase Functions, que proporcionará funciones como la autenticación y la gestión de datos.
- **Base de Datos:** La base de datos será Supabase (PostgreSQL), que almacenará información de eventos, artículos, preguntas y respuestas.
- **Almacenamiento:** Se utilizará Supabase Storage para manejar los archivos de los artículos y otros recursos multimedia.
- **Autenticación:** La autenticación de los usuarios se gestionará mediante Supabase Authentication.
- **Notificaciones:** Se utilizará Firebase Cloud Messaging para las notificaciones push.

3.2 Elección de Framework (Flutter)

- **Ventaja:** Flutter permite desarrollar interfaces modernas y atractivas con un único código base, facilitando un desarrollo ágil y consistente. Además, su enfoque declarativo y su amplio conjunto de widgets personalizados mejoran la experiencia del usuario y la productividad del desarrollador.
- **Desventaja:** Aunque Flutter ofrece un gran rendimiento, el tamaño de las aplicaciones puede ser mayor en comparación con soluciones nativas. Además, dado que solo se desarrollará para Android, no se aprovechará completamente su capacidad multiplataforma.
- **Trade-off:** Se eligió Flutter por su rapidez de desarrollo, interfaz flexible y rendimiento cercano al nativo. Aunque no se desarrollará para iOS, Flutter sigue siendo una opción eficaz para crear una aplicación Android moderna y mantenible.

3.3 Elección de Base de Datos (Supabase / PostgreSQL)

- **Ventaja:** Supabase, basado en PostgreSQL, proporciona una solución robusta y escalable para la gestión de datos relacionales, además de incluir autenticación y almacenamiento, lo que facilita la integración.

- **Desventaja:** Aunque PostgreSQL es poderoso, puede ser más complejo de manejar y menos flexible para datos no estructurados o muy dinámicos, comparado con bases de datos NoSQL como Firebase Firestore.
- **Trade-off:** Se optó por PostgreSQL debido a la necesidad de tener una base de datos relacional para gestionar las relaciones entre usuarios, eventos, artículos y preguntas. Sin embargo, tuvimos que sacrificar la flexibilidad de una base de datos NoSQL para facilitar consultas más dinámicas y escalables.

3.4 Elección de Autenticación (Supabase Authentication)

- **Ventaja:** Supabase Authentication es una solución fácil de integrar para gestionar el inicio de sesión de los usuarios con soporte para proveedores de autenticación social como Google, Facebook, etc.
- **Desventaja:** Dependencia de un servicio de terceros para la gestión de usuarios. Si Supabase experimenta problemas o cambios, tu aplicación podría verse afectada.
- **Trade-off:** Se decidió usar Supabase Authentication debido a su integración sencilla y a que ya está incluida como parte de Supabase. Aunque esto aumenta la dependencia de un tercero, los beneficios de la integración rápida superan el riesgo potencial de depender de un servicio externo.

3.5 Elección de Almacenamiento (Supabase Storage vs. Firebase Storage)

- **Ventaja:** Supabase Storage se integra bien con PostgreSQL y otras funcionalidades de Supabase, proporcionando una solución unificada para almacenamiento y bases de datos.
- **Desventaja:** Firebase Storage podría ofrecer un mejor rendimiento en términos de carga y distribución de archivos, especialmente si tienes una gran cantidad de usuarios distribuidos globalmente.
- **Trade-off:** Se optó por Supabase Storage para mantener la coherencia en la infraestructura y reducir la complejidad de integrar múltiples proveedores. Sin embargo, tuvimos que sacrificar el rendimiento potencial que Firebase Cloud Storage podría haber ofrecido en términos de latencia.

3.6 Elección de Servicios de Notificación (Firebase Cloud Messaging)

- **Ventaja:** Firebase Cloud Messaging (FCM) es fácil de integrar con aplicaciones móviles y permite enviar notificaciones push de manera eficiente.

- **Desventaja:** Utilizar Firebase implica estar atado a los servicios de Google y podría complicar la transición a otro servicio si se requiere en el futuro.
- **Trade-off:** Elegimos Firebase Cloud Messaging debido a su facilidad de integración y a su soporte extendido. Sin embargo, esto aumenta la dependencia de los servicios de Google, lo que podría ser un problema si decidieramos migrar a una plataforma diferente en el futuro.

3.7 Elección de Infraestructura Backend (*Supabase Functions vs. Backend Personalizado*)

- **Ventaja:** Usar Supabase Functions para la lógica de backend permite escalar rápidamente sin necesidad de gestionar infraestructura compleja.
- **Desventaja:** Usar funciones en la nube limita el control sobre la infraestructura, y podría ser menos flexible que una solución de backend personalizada.
- **Trade-off:** Decidimos usar Supabase Functions por su facilidad de integración y capacidad de escalar rápidamente sin necesidad de gestionar servidores. Sin embargo, esto significa perder algo de flexibilidad que tendríamos con un backend completamente personalizado.

3.8 Elección de Patrón de Diseño: MVVM (Model-View-ViewModel)

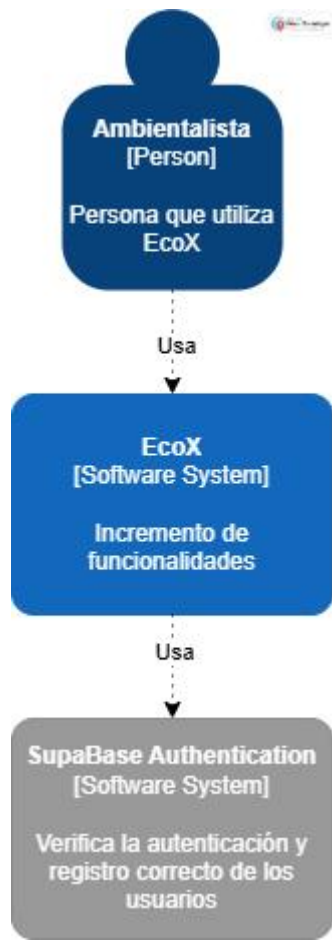
- **Ventaja:**
La arquitectura en capas permite separar el sistema en niveles con responsabilidades bien definidas: la capa de modelos (estructuras de datos y su serialización), la capa de servicios (acceso a datos y lógica de comunicación con el backend) y la capa de vistas (interfaz de usuario). Esta separación promueve un desarrollo organizado y facilita el mantenimiento, ya que un cambio en el acceso a datos —por ejemplo, en la integración con Supabase— queda contenido en la capa de servicios sin afectar directamente a la interfaz. Al aislar la lógica de acceso a datos detrás de la capa de servicios, la interfaz no depende de los detalles del backend, lo que mejora la modularidad y la reutilización de código.
- **Desventaja:**
La principal desventaja de este enfoque es que la lógica de presentación reside dentro de las propias vistas, que gestionan su estado localmente. Si no se mantiene disciplina en la separación, las vistas pueden concentrar demasiada responsabilidad y volverse difíciles de mantener a medida que crecen. Asimismo, al no existir una capa intermedia dedicada a la lógica de presentación, las pruebas unitarias sobre dicha lógica resultan menos directas que en patrones que la aíslan por completo de la interfaz gráfica.
- **Trade-off (Compensación):**
A pesar de que otros patrones ofrecen un mayor desacoplamiento entre la interfaz y la lógica de presentación, se optó por una arquitectura en capas por su menor

complejidad y su curva de aprendizaje más accesible, lo cual se ajusta al alcance del proyecto, a los tiempos de desarrollo por iteraciones y a un equipo con responsabilidades divididas. Esta decisión mantiene una separación clara entre el acceso a datos y la interfaz mediante la capa de servicios, mientras acepta como compensación que la lógica de presentación permanezca en las vistas. De este modo se prioriza un desarrollo más ágil y sencillo de comprender por todo el equipo, sin renunciar a los beneficios de mantenibilidad que aporta la organización en capas.

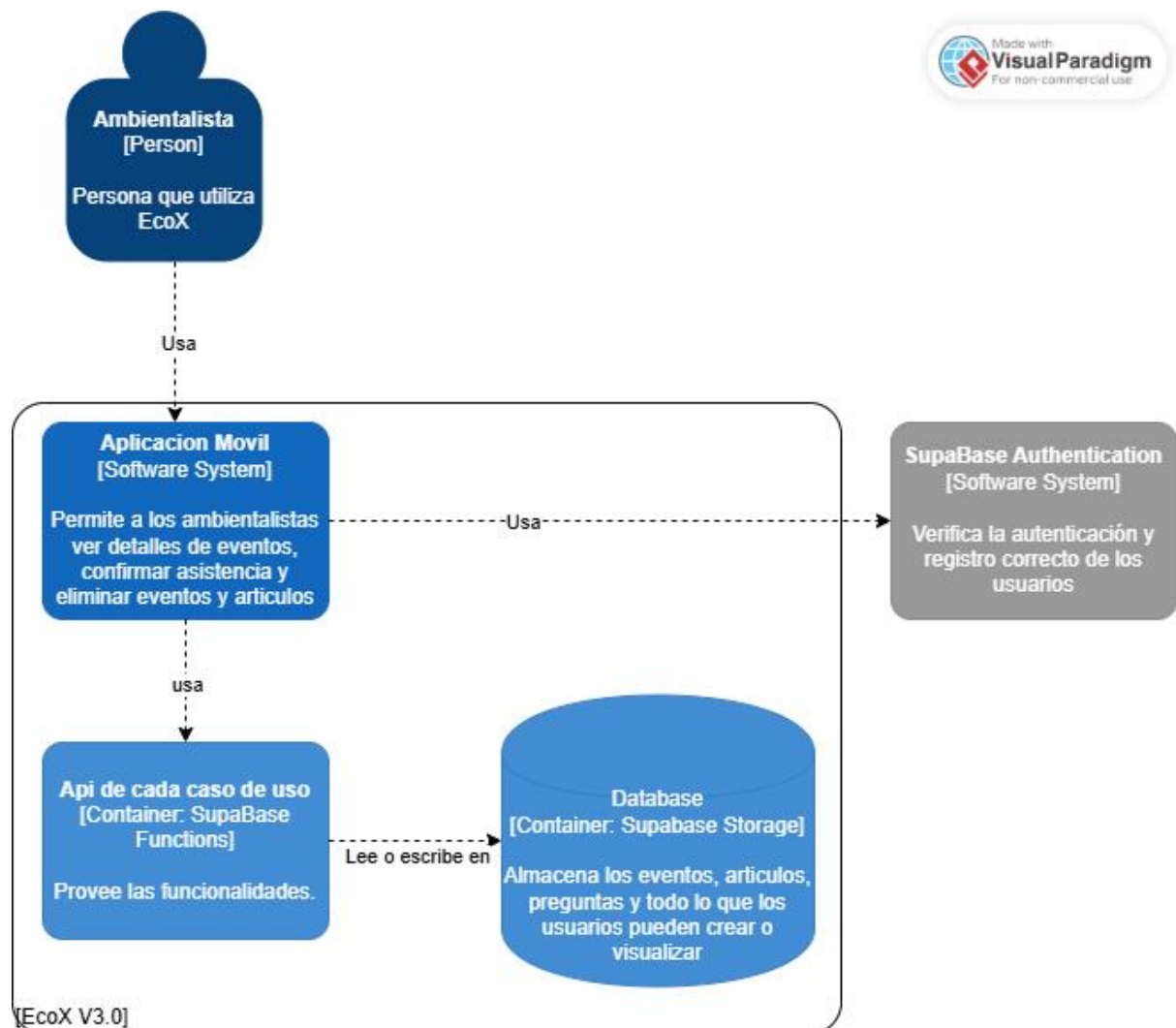
3.9 Elección de Escalabilidad (Base de Datos Relacional vs. No Relacional)

- **Ventaja:** Una base de datos relacional como PostgreSQL (a través de Supabase) proporciona integridad de datos y facilita la realización de consultas complejas, que son necesarias para tu aplicación, que involucra múltiples relaciones entre usuarios, eventos y artículos.
- **Desventaja:** Las bases de datos no relacionales, como Firebase Firestore, podrían ser más fáciles de escalar y manejar con estructuras de datos más flexibles y sin necesidad de definir esquemas estrictos.
- **Trade-off:** Optamos por PostgreSQL para asegurar una estructura de datos consistente y relaciones entre entidades, pero sacrificamos la flexibilidad y la capacidad de escalabilidad que una base de datos NoSQL podría ofrecer.

3.11 diagrama de contexto



3.12 diagrama de contenedores



4. Diseño de la Base de Datos

4.1 Modelo de Datos

Entidades y Propiedades:

- **Usuario**
 - ID de Usuario (PK)
 - Nombre
 - Correo Electrónico

- Teléfono
- Ubicación
- Historial de Participación en Actividades

- **Actividad de Conservación**

- ID de Actividad (PK)
- Nombre de la Actividad
- Descripción
- Ubicación
- Fecha y Hora
- Tipo de Actividad (Presencial/Virtual)
- Disponibilidad de Cupos
- Materiales Requeridos
- Url Image
- Tipo de Categoría
- Organizador (FK - Usuario o Asociación)

- **Artículos en Desuso**

- ID de Artículo (PK)
- Nombre del Artículo
- Descripción
- Estado (Usado, Como Nuevo, Nuevo)
- Tipo de Categoría
- Precio
- Ubicación
- ID de Usuario (FK - Propietario)

- Fotos del Artículo

- **Transacciones en el Marketplace**

- ID de Transacción (PK)
- ID de Comprador (FK - Usuario)
- ID de Vendedor (FK - Usuario)
- ID de Artículo (FK)
- Estado de la Transacción (Pendiente, Completado, Cancelado)
- Fecha de la Transacción
- Modalidad de Entrega (Presencial, Envío)

- **Foro de Preguntas y Respuestas**

- **Preguntas**

- ID de Pregunta (PK)
- Pregunta
- Fecha de Publicación
- ID de Usuario (FK - Autor de la pregunta)
- Nombre de Usuario
- Total de respuestas

- **Respuestas**

- ID de Respuesta (PK)
- Respuesta
- ID de Pregunta
- ID de Usuario (FK - Autor de la respuesta)
- Nombre de usuario

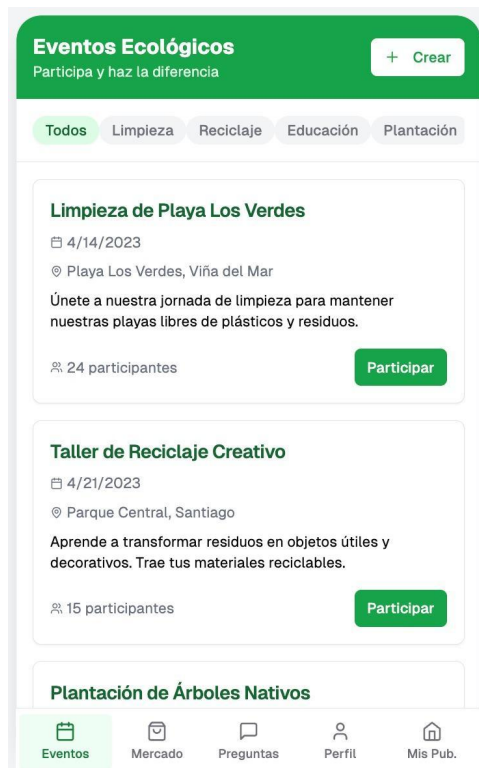
Relaciones entre Entidades:

- **Usuario - Actividad de Conservación:** Un usuario puede participar en múltiples actividades.
- **Usuario - Transacciones:** Un usuario puede vender, comprar o intercambiar artículos.
- **Usuario - Foro:** Un usuario puede hacer preguntas y responderlas en el foro.
- Diagrama de clases

5. Diseño de la Interfaz de Usuario

5.1 Pantallas Principales

- Pantalla de eventos: Listar eventos disponibles con la opción de crear uno nuevo.



- pantalla crear evento

Crear Nuevo Evento



Completa los detalles para crear un nuevo evento ecológico.

Título del Evento

Ej: Limpieza de Playa

Fecha

dd/mm/yyyy



Ubicación

Ej: Parque Central, Santiago

Categoría

Selecciona una categoría

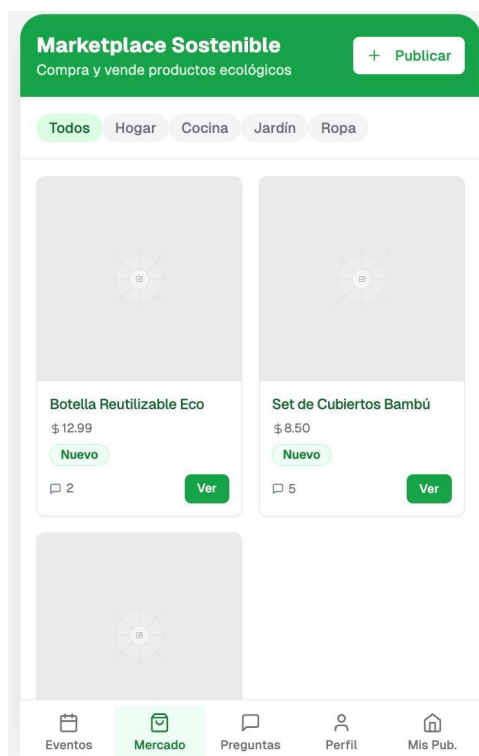


Descripción

Describe los detalles del evento...

Crear Evento

- Pantalla de artículos: Mostrar artículos ecológicos



- pantalla crear artículo

Publicar Artículo



Completa los detalles para publicar un nuevo artículo en el marketplace.

Título del Artículo

Ej: Botella Reutilizable

Precio (\$)

Ej: 15.99

Estado

Selecciona el estado



Categoría

Selecciona una categoría



Descripción

Describe tu artículo...

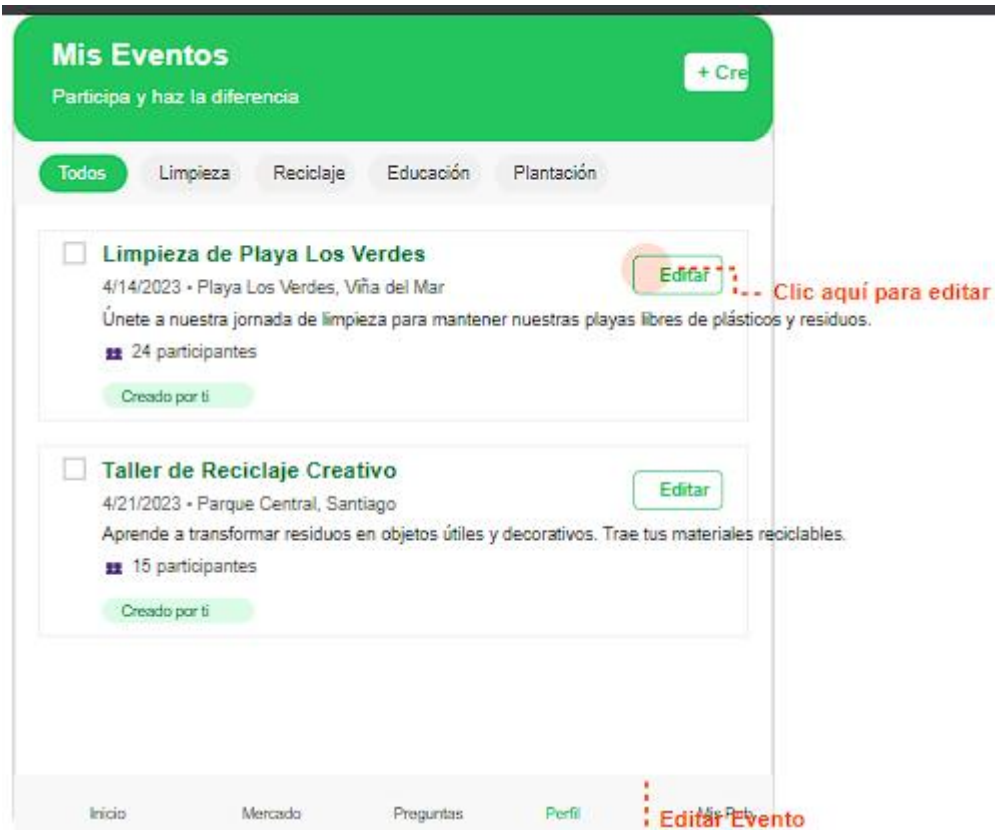
Publicar Artículo

- pantalla de mis publicaciones



5.2 Pantallas segunda iteración

- Pantalla editar Evento



Editar Evento

Título del Evento

Limpeza de Playa Los Verdes

Fecha

14/04/2023

Ubicación

Playa Los Verdes, Viña del Mar

Categoría

Limpeza

Descripción

Únete a nuestra jornada de limpieza para mantener nuestras playas libres de plásticos y residuos.

Eliminar

Guardar Cambio

- Pantalla editar artículo

Mis Productos

Productos que has publicado

+ Publicar

Todos

Hogar

Cocina

Jardín

Ropa

Botella Reutilizable Eco

\$12.99

2 visitas

Editar

Set de Cubiertos Bambú

\$8.50

Editar

Inicio

Editar Producto

Mis Productos

Preguntas

Perfil

Mis Publicaciones

... Clic aquí para editar

Editar Producto

Editar Producto

Título del Producto

Botella Reutilizable Eco

Precio (CLP)

12.99

Categoría

Hogar

Estado

Nuevo

Imágenes

Agregar imagen

Eliminar

Guardar Cambio

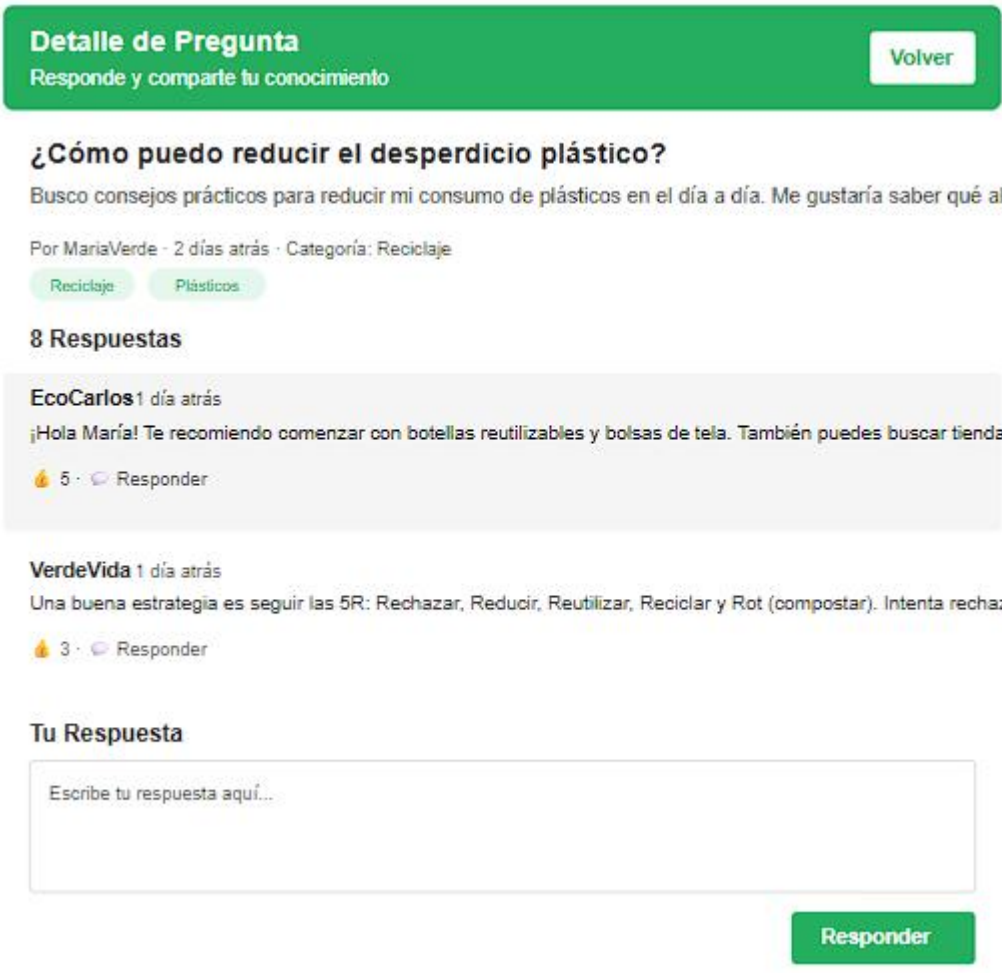
- **Publicar pregunta**



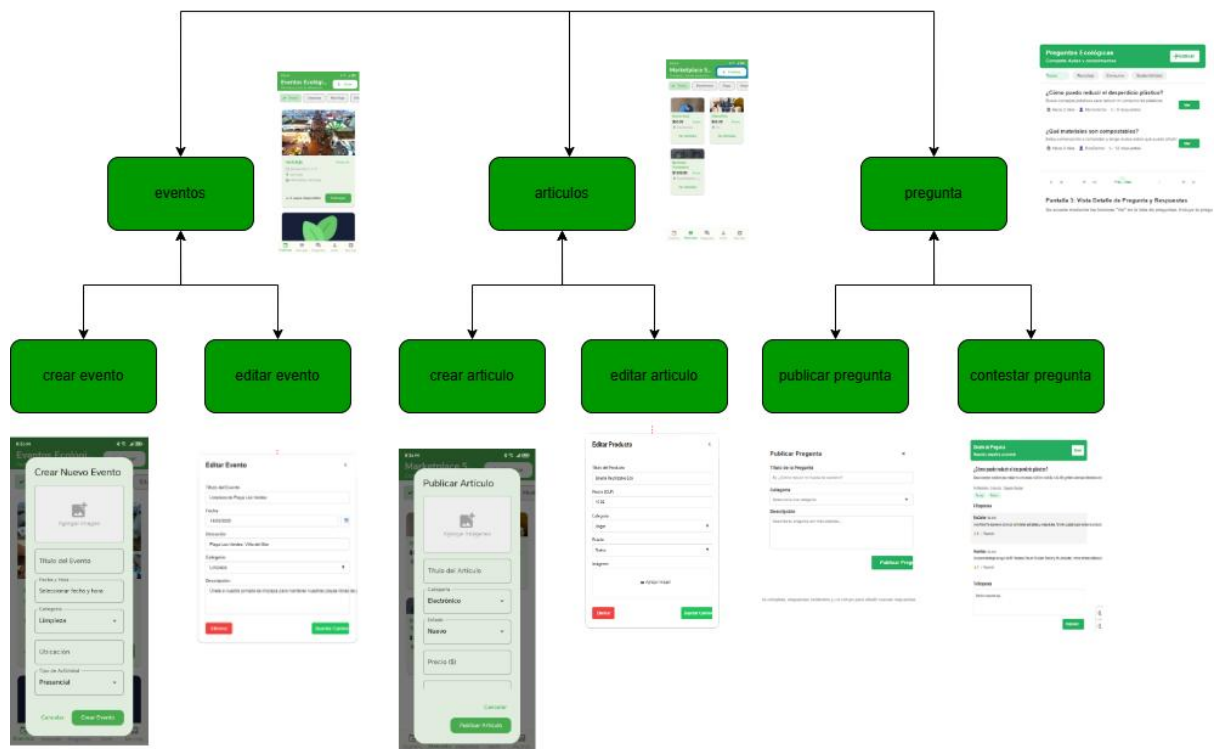
Pantalla 3: Vista Detalle de Pregunta y Respuestas

Se accede mediante los botones "Ver" en la lista de preguntas. Incluye la pregunta completa, respuestas existentes y un campo para añadir nuevas respuestas.

- **Contestar pregunta**

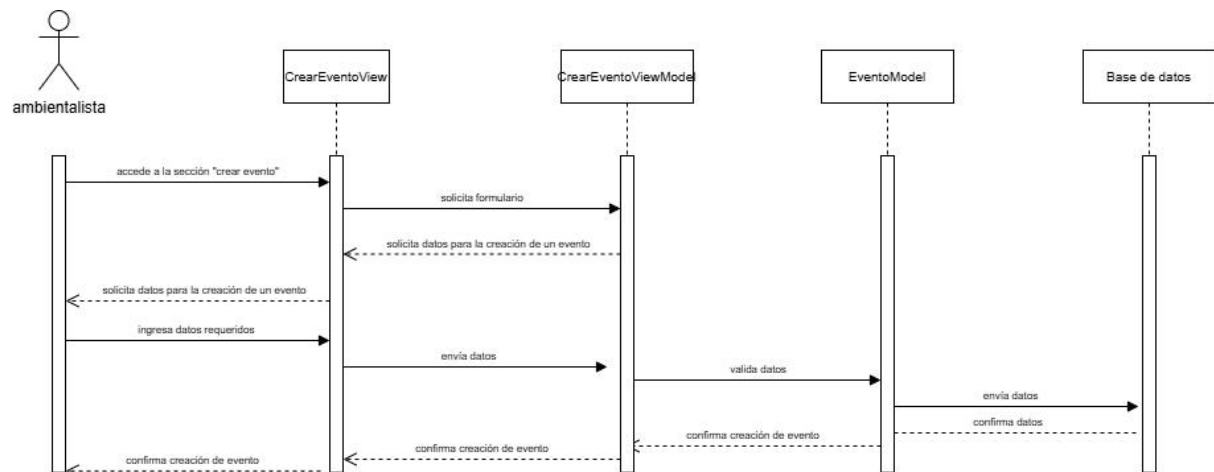


- mapa de navegación

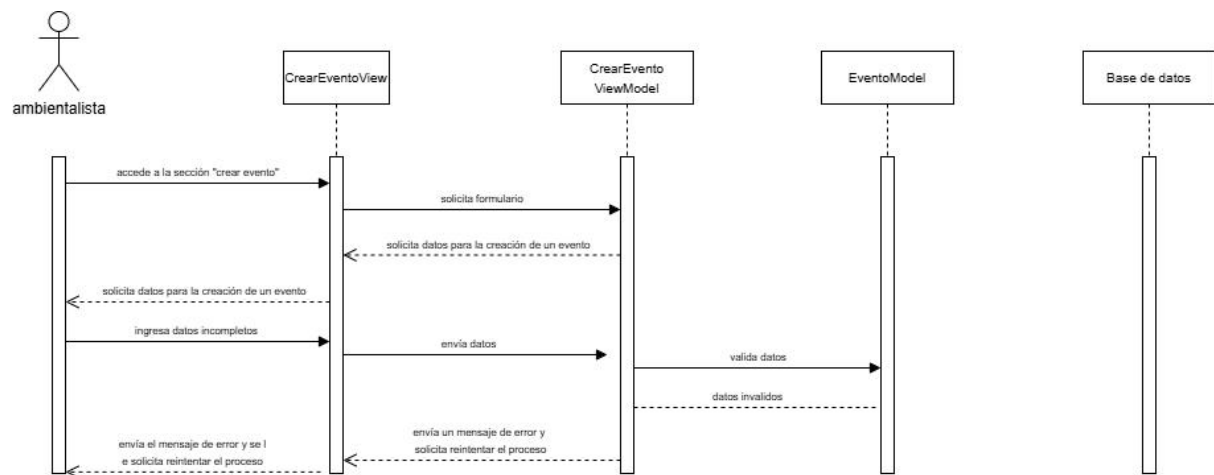


6. Diagramas de secuencia

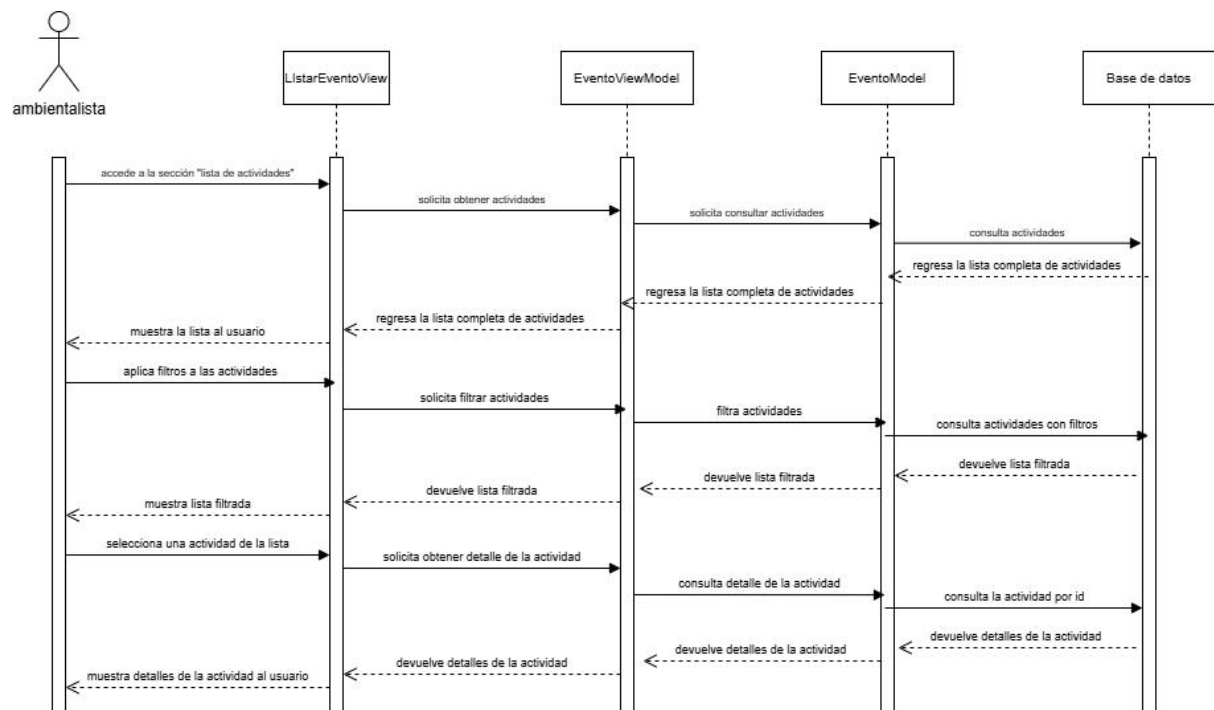
6.1 Crear evento



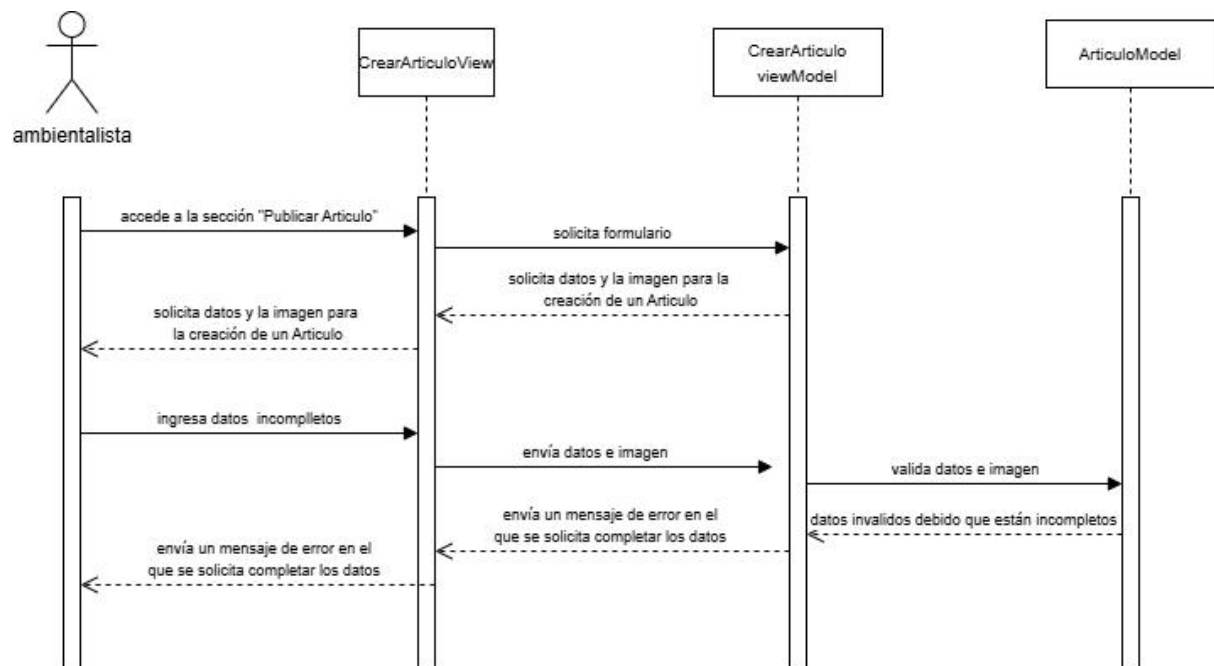
6.2 crear evento flujo alternativo



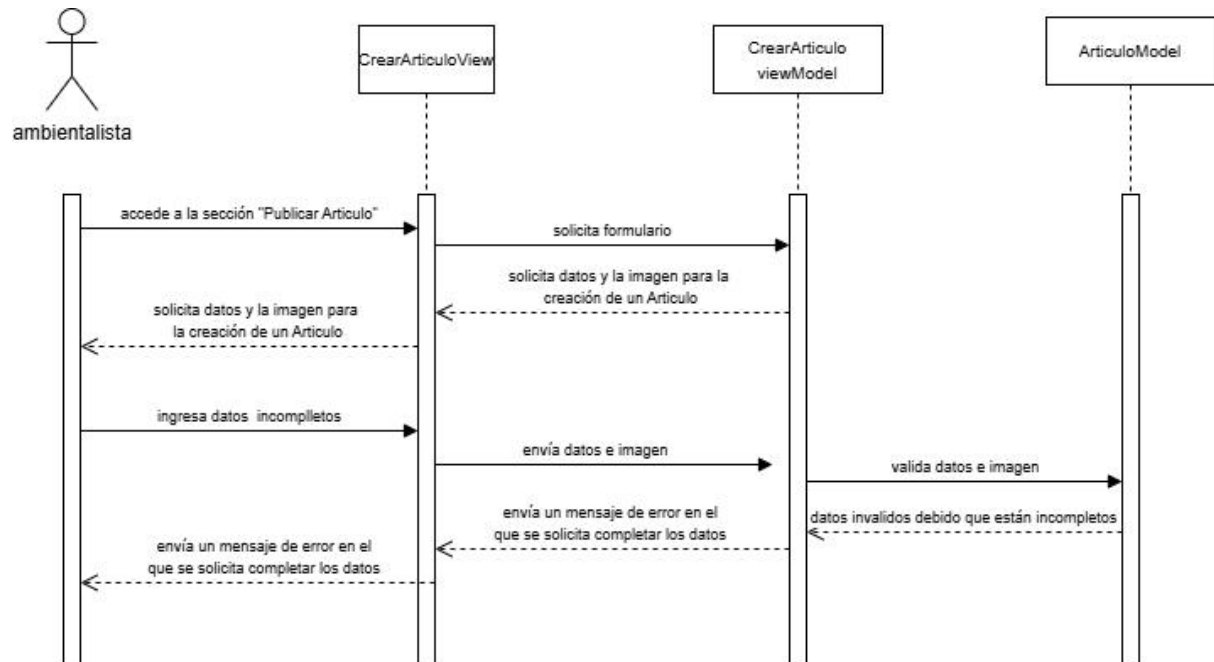
6.3 listar evento



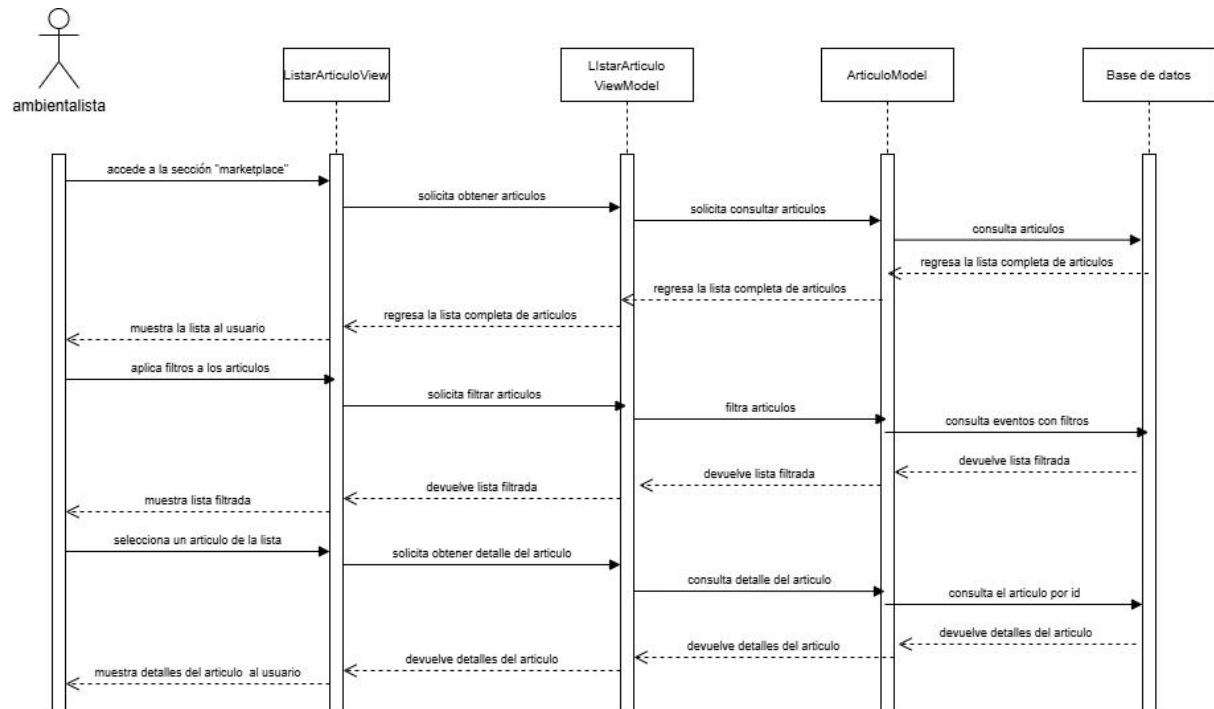
6.4 crear artículo



6.5 crear artículo flujo alternativo

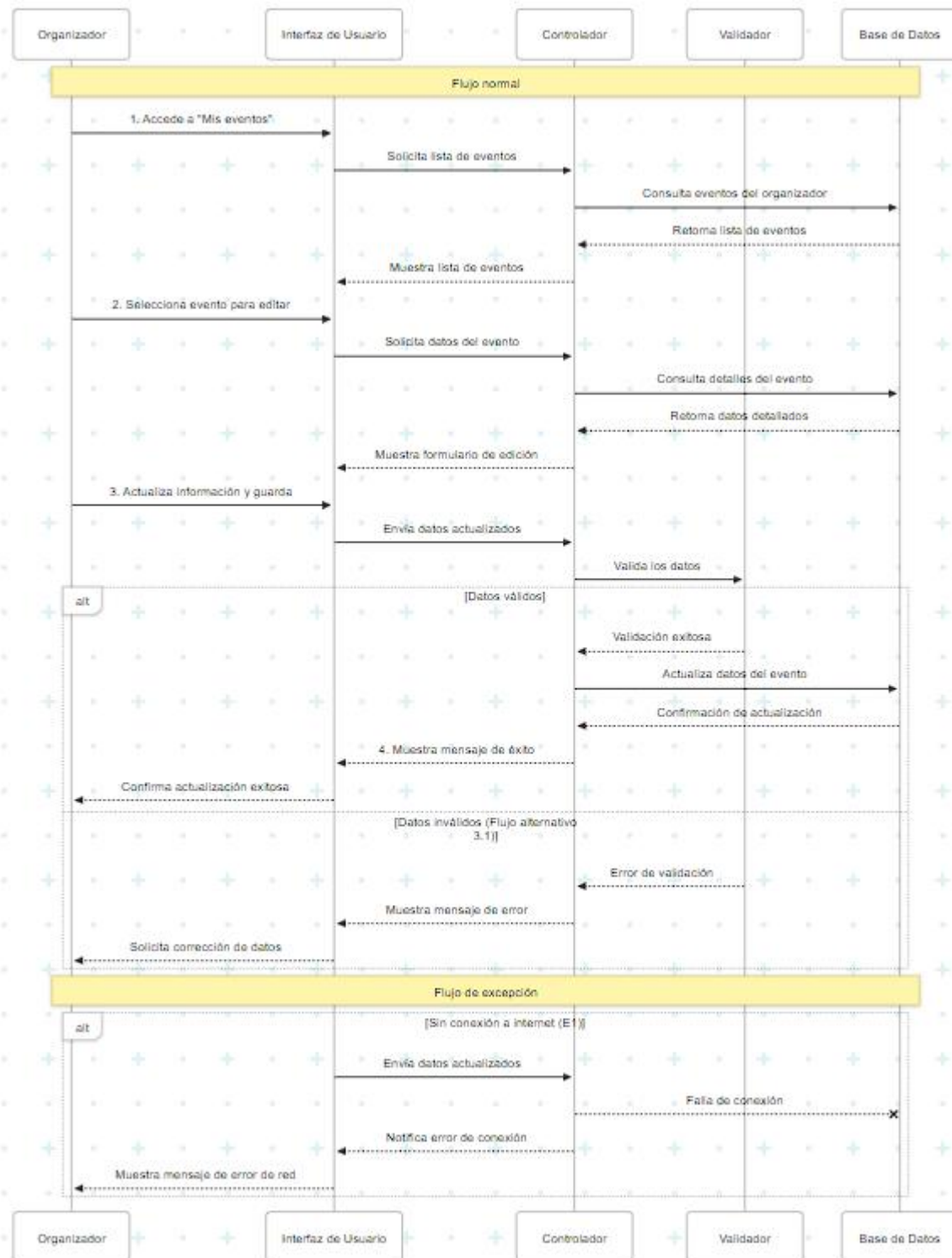


6.6 listar artículo

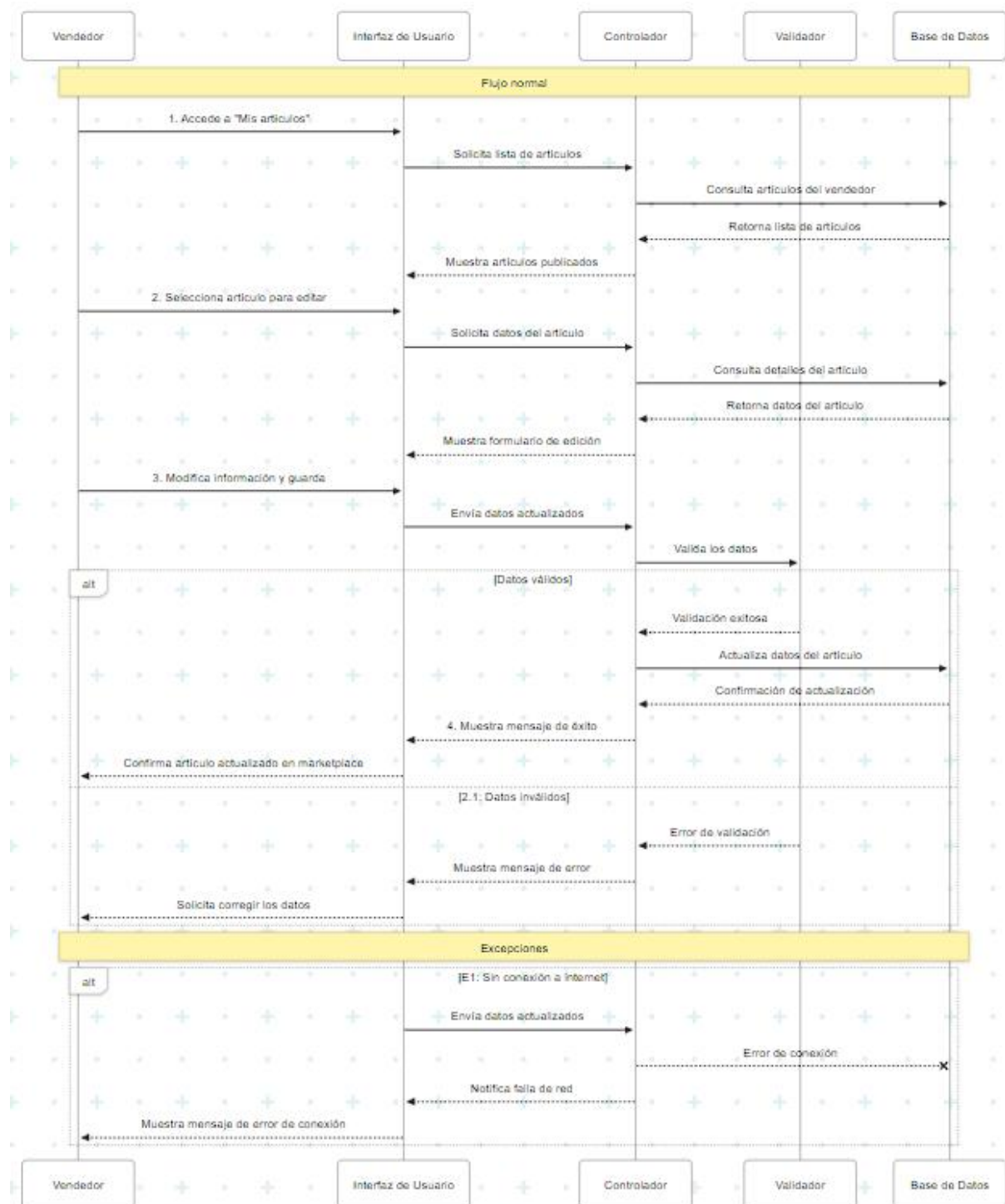


7. Diagramas de secuencia 2da iteración

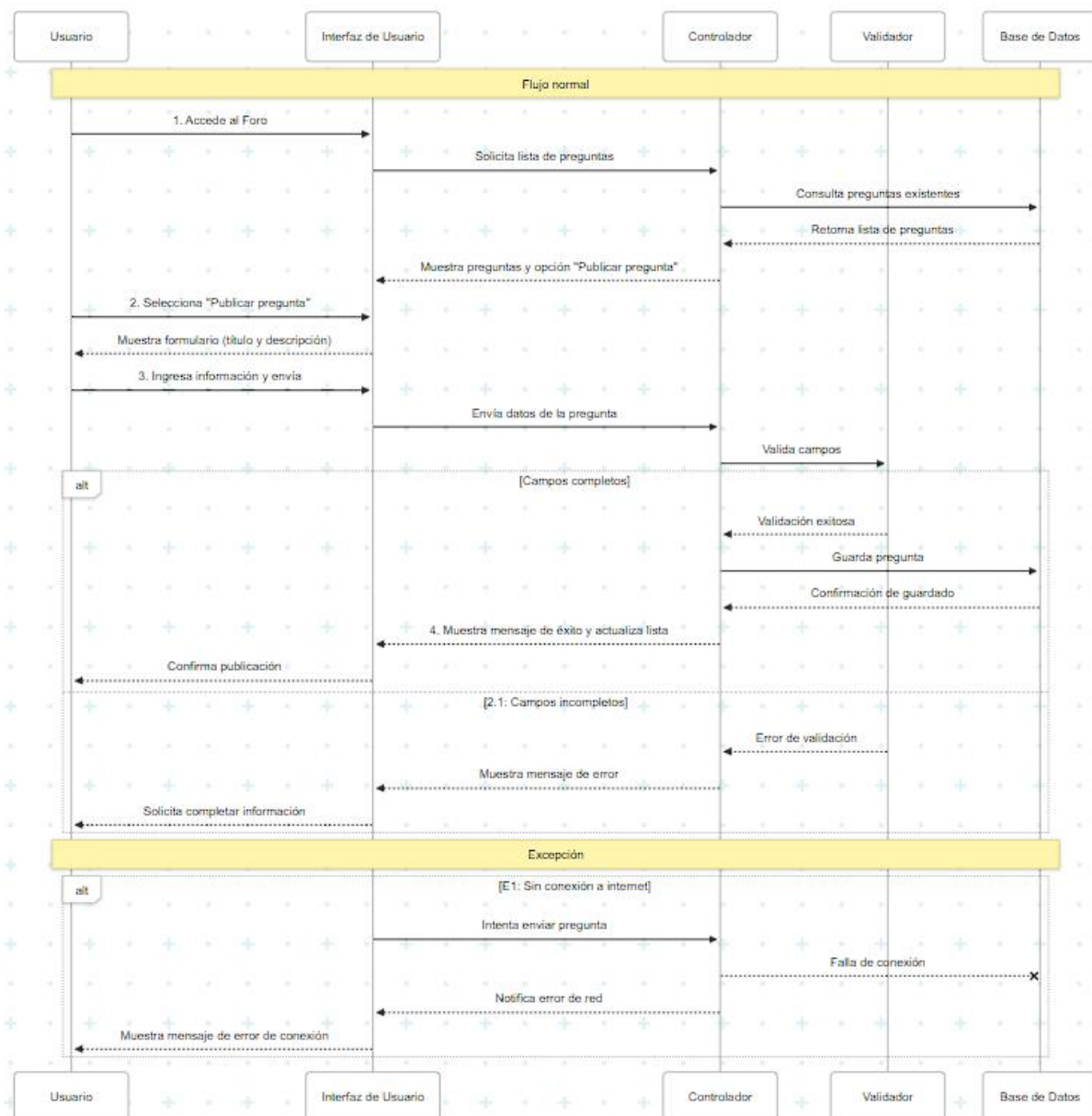
7.1 Editar eventos



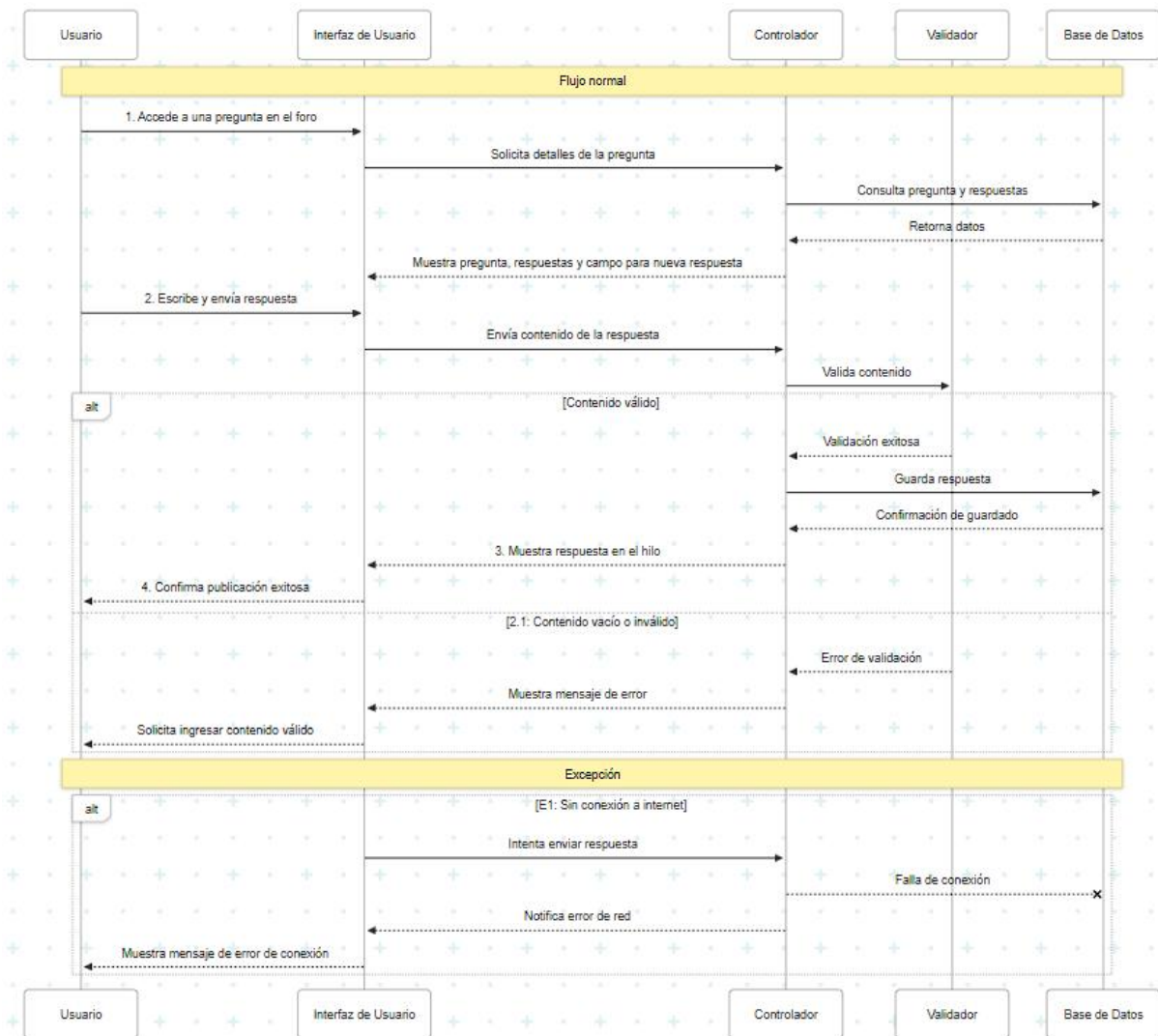
7.2 Editar artículos



7.3 Hacer pregunta

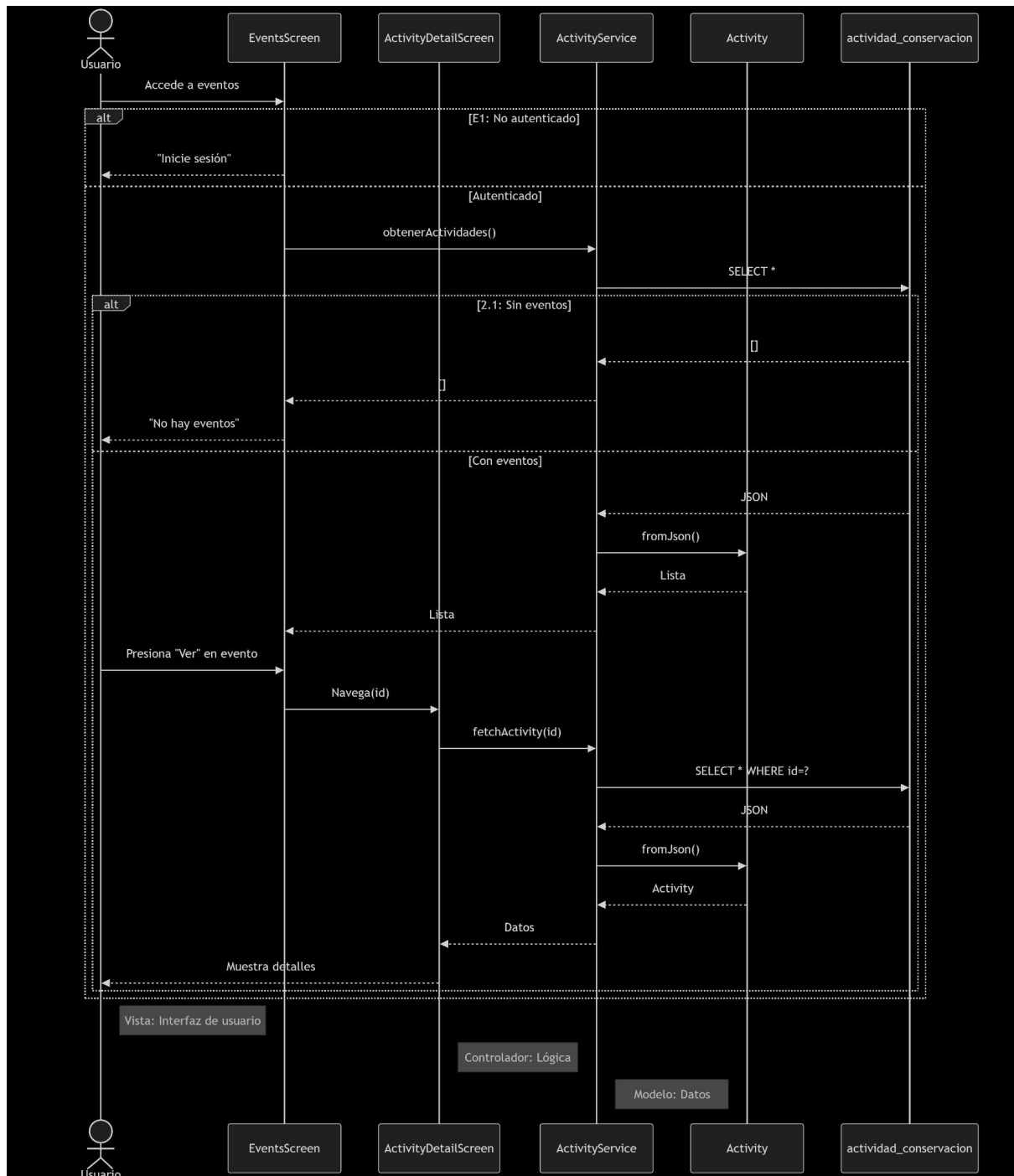


7.4 Responder pregunta

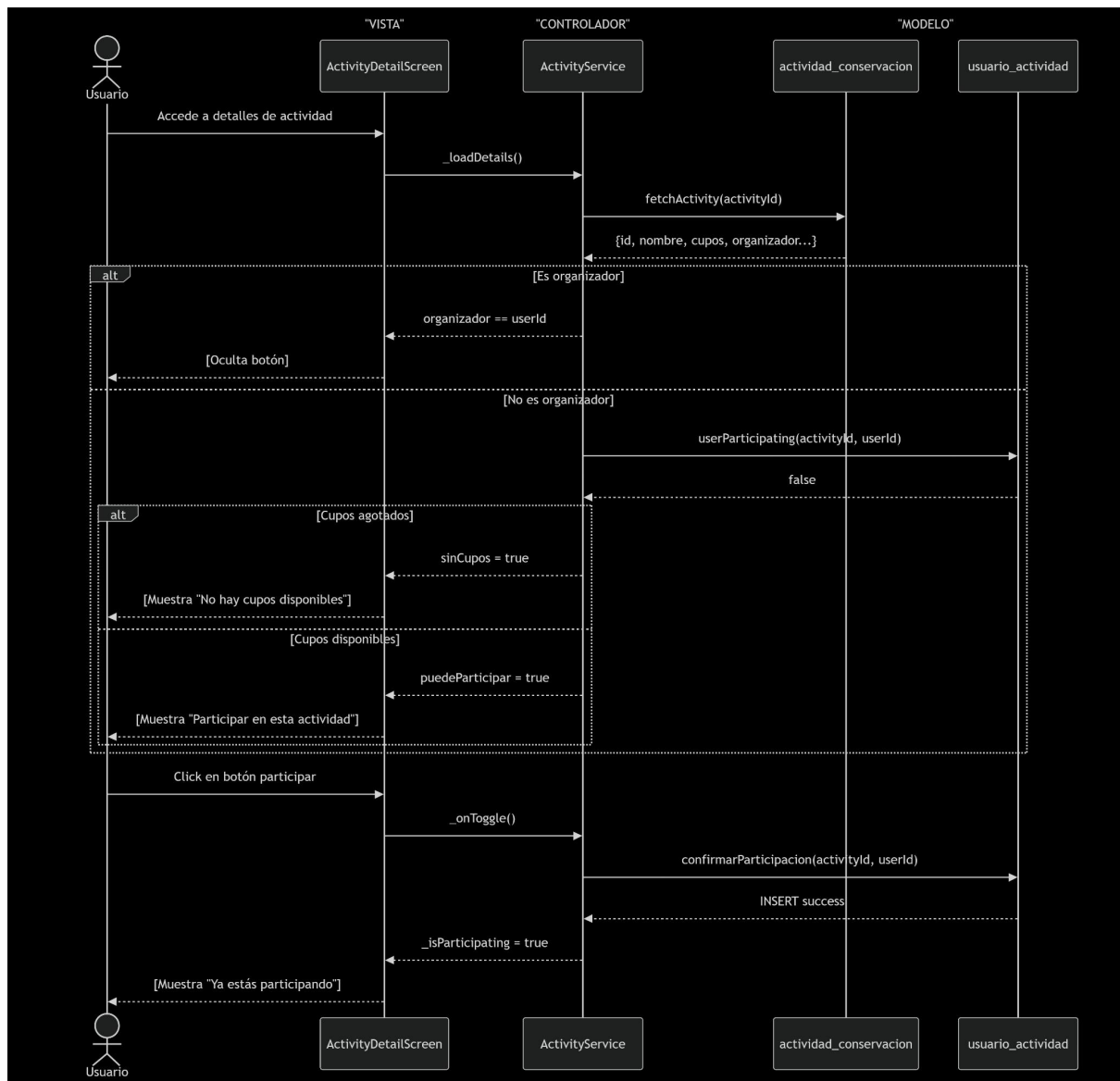


8. Diagramas de secuencia 3er iteración

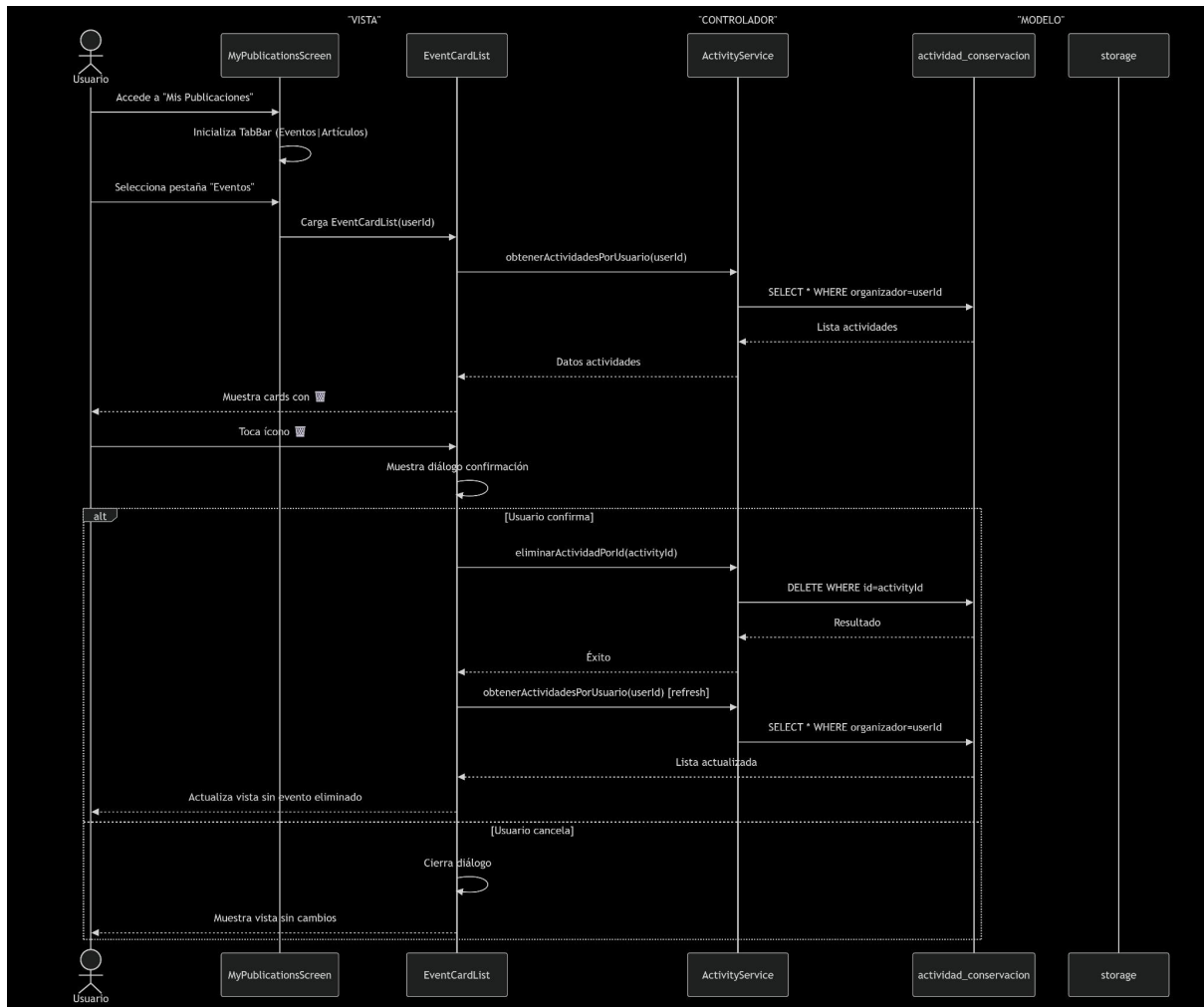
8.1 Visualizar detalles de actividad



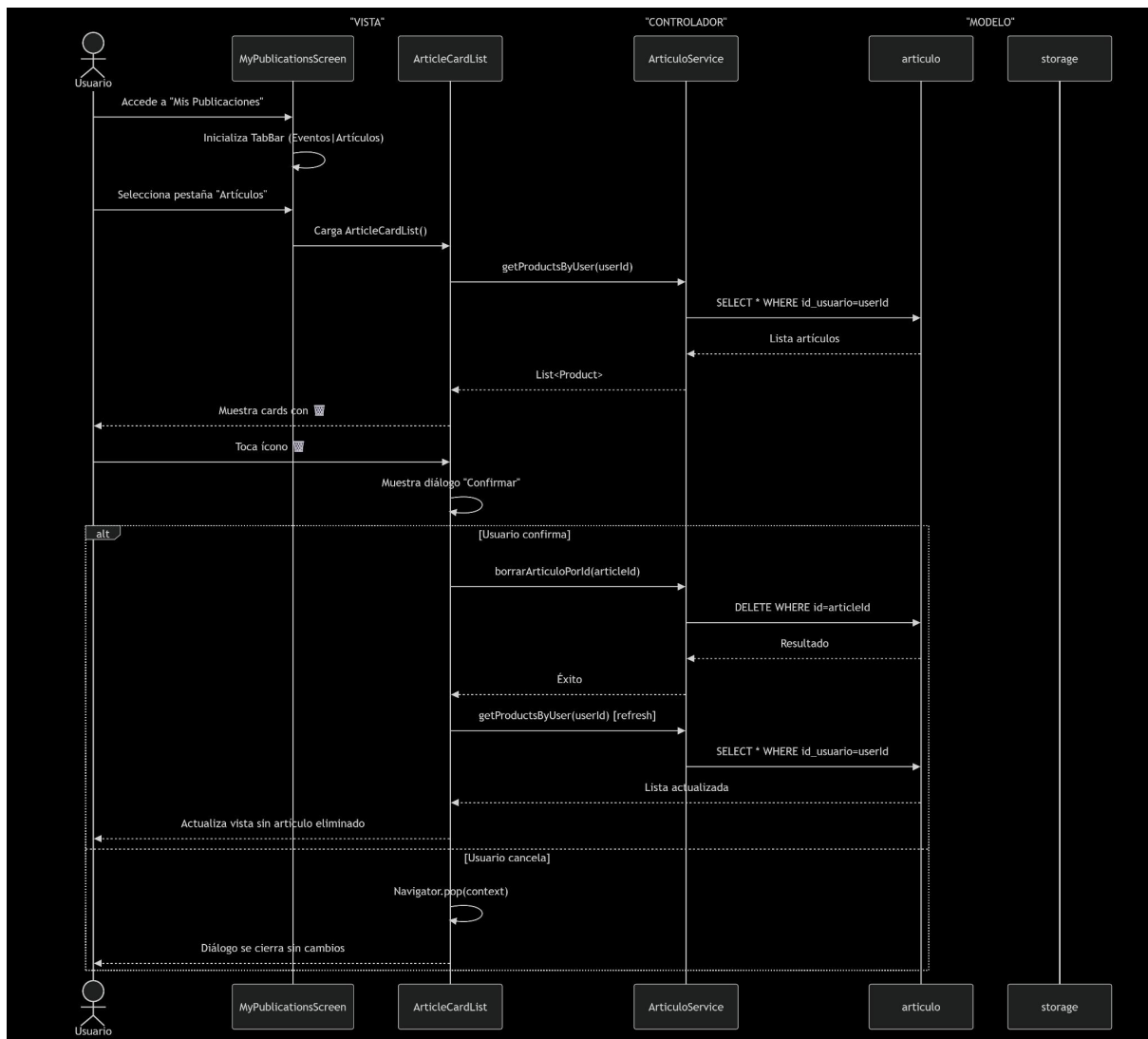
8.2 Registrar asistencia a evento



8.3 Eliminar evento

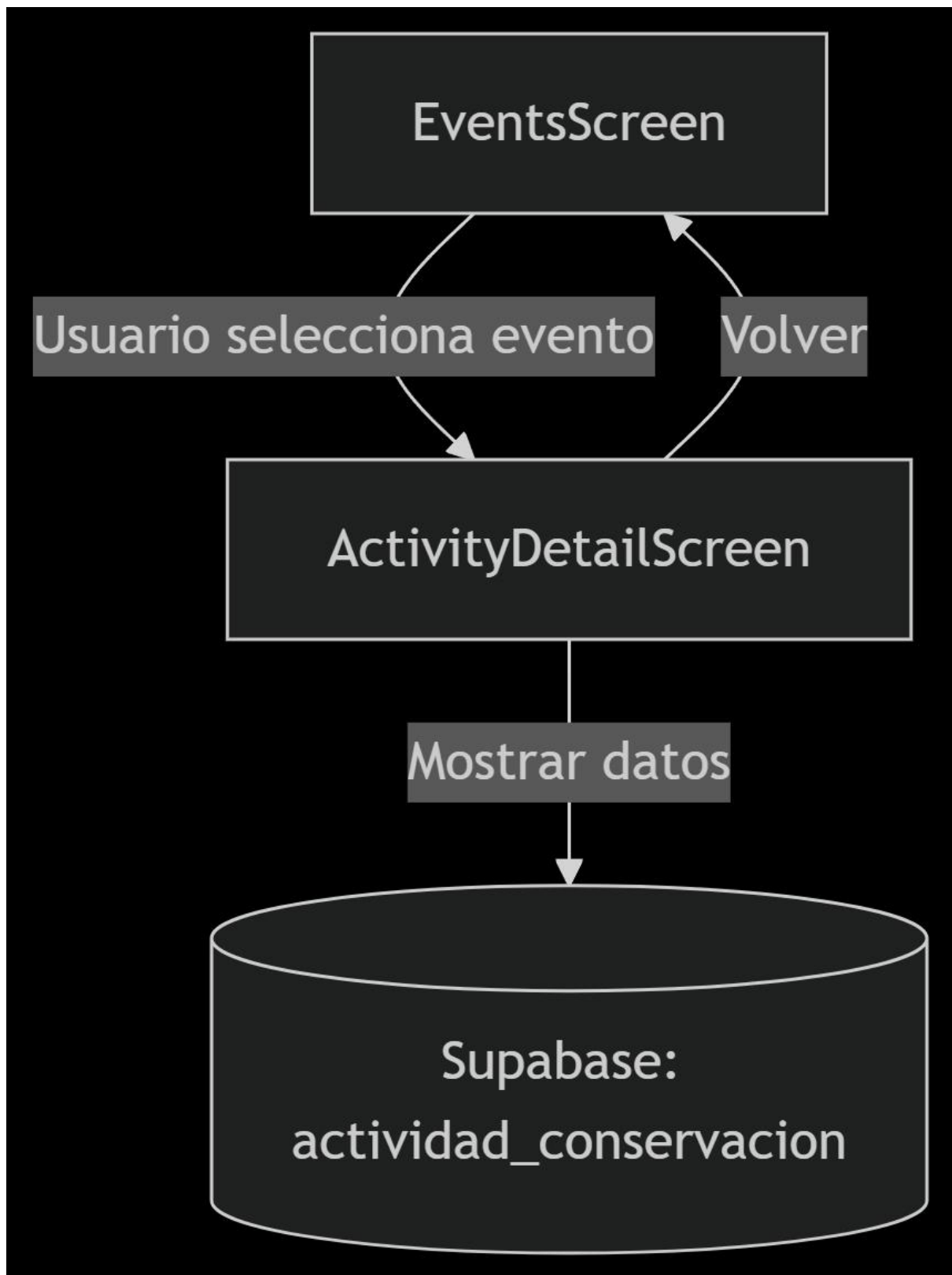


8.4 Eliminar producto

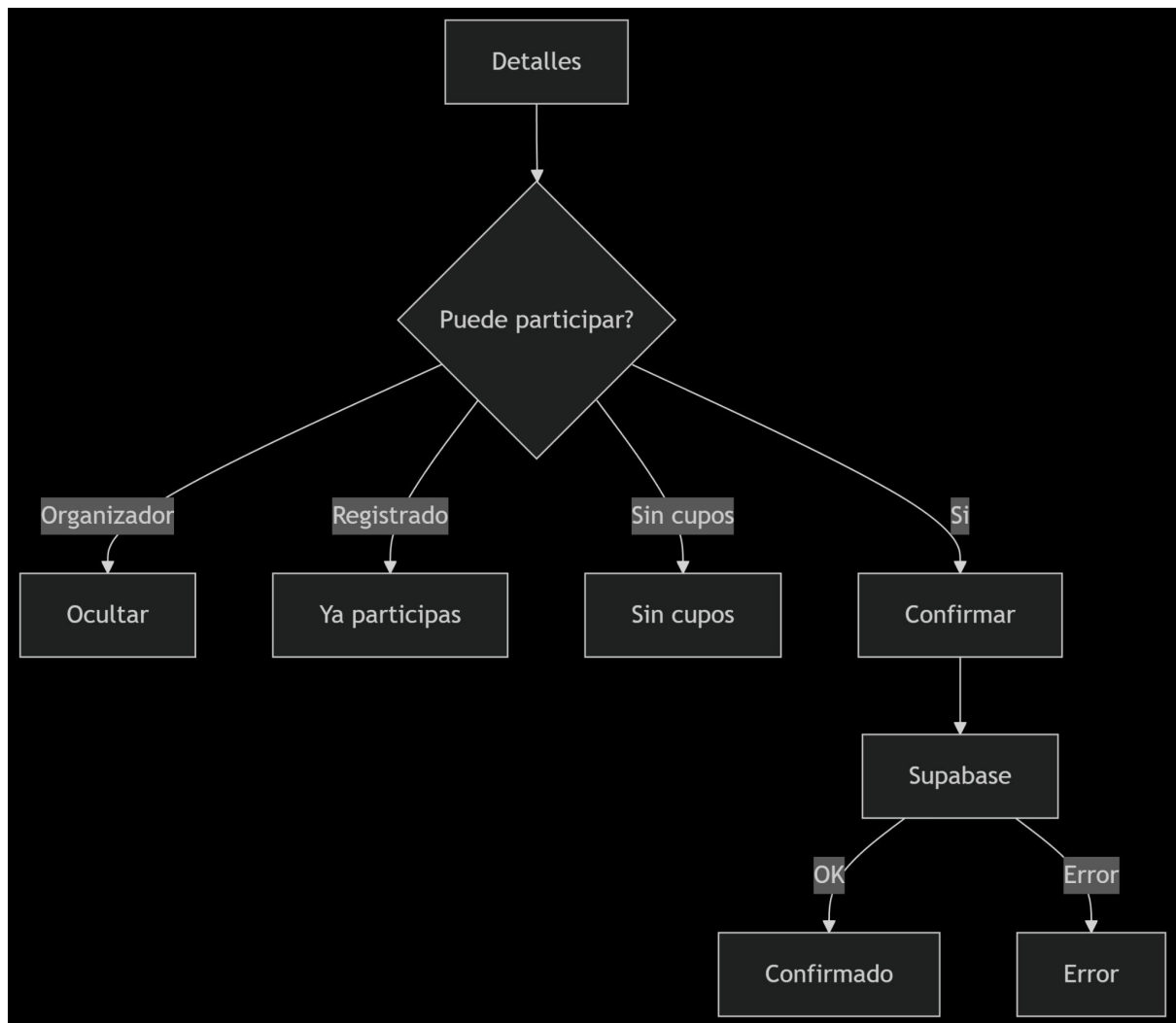


9. Mapas de navegación

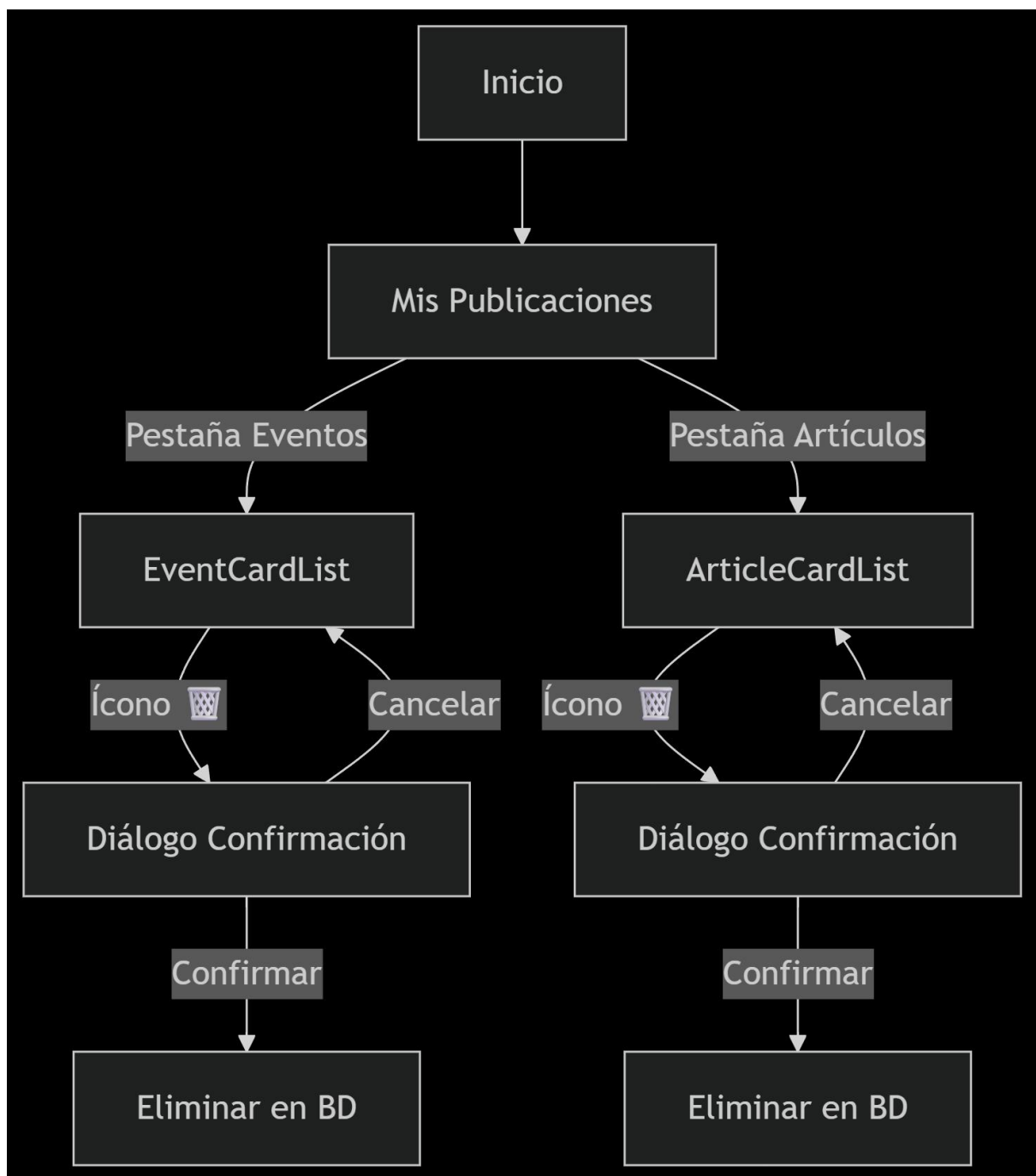
9.1 Caso de uso 9 - Visualizar detalles de evento



9.2 Caso de uso 10 - Registrar asistencia a evento

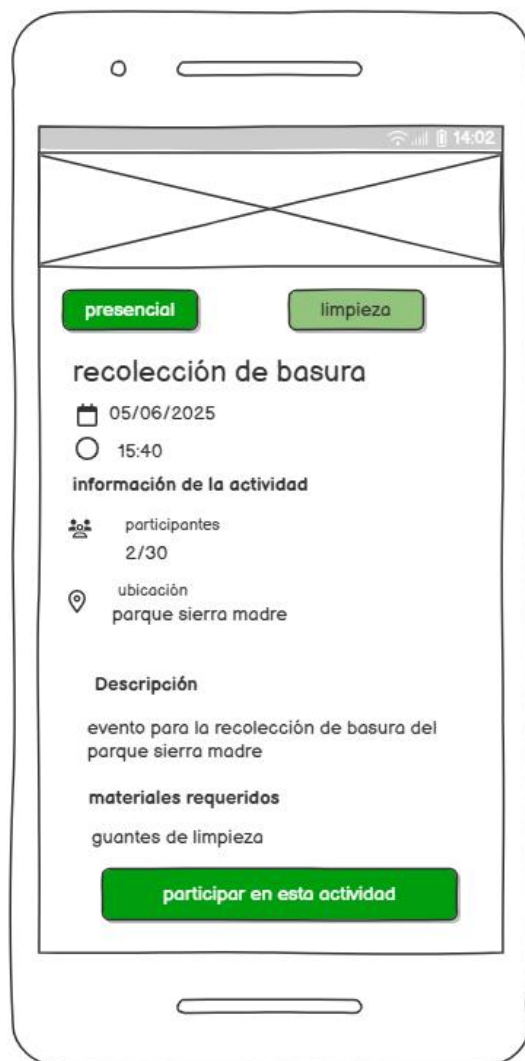


9.3 Caso de uso 11 y 12 - Eliminar evento y eliminar producto



10. Mock ups

10.1 Caso de uso 9 - Visualizar detalles de evento



10.2 Caso de uso 10 - Registrar asistencia a evento



10.3 Caso de uso 11 - Eliminar evento



10.4 Caso de uso 12 - Eliminar artículo

